

RECURSION:**SUM OF N NUMBERS:**

```
public class Main
{
    static int sum(int n)
    {
        if (n == 0)
            return 0;
        return n + sum(n - 1);
    }
    public static void main(String[] args)
    {
        System.out.println(sum(5));
    }
}
```

OUTPUT:

15

SUM OF DIGITS:

```
import java.util.*;
public class Main
{
    static int sumOfDigits(int n)
    {
        if (n == 0)
            return 0;
        return (n % 10) + sumOfDigits(n / 10);
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println("Sum of digits: " + sumOfDigits(n));
    }
}
```

OUTPUT:

Enter a number: 204623798

Sum of digits: 41

FACTORIAL:

```
public class Main
{
```

```

static int fact(int n)
{
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}
public static void main(String[] args)
{
    System.out.println(fact(7));
}
}

```

OUTPUT:

5040

FIBONACCI:

```

import java.util.*;
public class Main
{
    static int fibonacci(int n)
    {
        if (n == 0)
            return 0;
        if (n == 1)
            return 1;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        System.out.println("Fibonacci Number: " + fibonacci(n));
    }
}

```

OUTPUT:

Enter n: 6
Fibonacci Number: 8

MERGE SORT:

```

import java.util.*;
public class Main {
    public static void merge(int[] arr, int left, int mid, int right) {

```

```

int n1 = mid - left + 1;
int n2 = right - mid;
int[] L = new int[n1];
int[] R = new int[n2];
for (int i = 0; i < n1; i++)
{
    L[i] = arr[left + i];
}
for (int j = 0; j < n2; j++)
{
    R[j] = arr[mid + 1 + j];
}
int i = 0, j = 0, k = left;
while (i < n1 && j < n2)
{
    if (L[i] <= R[j])
    {
        arr[k++] = L[i++];
    }
    else
    {
        arr[k++] = R[j++];
    }
}
while (i < n1)
{
    arr[k++] = L[i++];
}
while (j < n2)
{
    arr[k++] = R[j++];
}
}
public static void mergeSort(int[] arr, int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
public static void main(String[] args)

```

```

{
    int[] arr = {5, 2, 4, 6, 1, 3};
    mergeSort(arr, 0, arr.length - 1);
    System.out.println("Sorted Array:");
    for (int x : arr) {
        System.out.print(x + " ");
    }
}
}

```

OUTPUT:

Sorted Array:
1 2 3 4 5 6

QUICK SORT:

```

import java.util.*;
public class Main
{
    public static int partition(int[] arr, int low, int high)
    {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++)
        {
            if (arr[j] < pivot)
            {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }
    public static void quickSort(int[] arr, int low, int high)
    {
        if (low < high)
        {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }
}

```

```
    }  
}  
public static void main(String[] args)  
{  
    int[] arr = {5, 2, 4, 6, 1, 3};  
    quickSort(arr, 0, arr.length - 1);  
    System.out.println("Sorted Array:");  
    for (int x : arr)  
    {  
        System.out.print(x + " ");  
    }  
}  
}
```

OUTPUT:

Sorted Array:

1 2 3 4 5 6